Introduction

- A computer instruction is made up of an operation code (op-code) followed by either zero, one or two bytes of operands
- The op-code identifies the type of operation to be performed while the operands identify the source and destination of the data
- The operand can be:
  - The data value itself
  - A CPU register
  - A memory location
  - An I/O port
- If the instruction is associated with more than one operand, the format is always:

  Instruction Destination, Source

Instruction Types

- The C8051F020 instructions are divided into five functional groups:
  - Arithmetic operations
  - Logical operations
  - Data transfer operations
  - Boolean variable operations
  - Program branching operations

Arithmetic Operations

- [@Ri] implies contents of memory location pointed to by R0 or R1
- Rn refers to registers R0-R7 of the currently selected register bank

Mnemonic	Description
ADD A, Rn	A = A + [Rn]
ADD A, direct	A = A + [direct memory]
ADD A,@Ri	A = A + [memory pointed to by Ri]
ADD A,#data	A = A + immediate data
ADDC A,Rn	A = A + [Rn] + CY
ADDC A, direct	A = A + [direct memory] + CY
ADDC A,@Ri	A = A + [memory pointed to by Ri] + CY
ADDC A,#data	A = A + immediate data + CY
SUBB A,Rn	A = A - [Rn] - CY
SUBB A, direct	A = A - [direct memory] - CY
SUBB A,@Ri	A = A - [@Ri] - CY
SUBB A,#data	A = A - immediate data - CY
INC A	A = A + 1
INC Rn	[Rn] = [Rn] + 1
INC direct	[direct] = [direct] + 1
INC @Ri	[@Ri] = [@Ri] + 1
DEC A	A = A - 1
DEC Rn	[Rn] = [Rn] - 1
DEC direct	[direct] = [direct] - 1
DEC @Ri	[@Ri] = [@Ri] - 1
MUL AB	Multiply A & B
DIV AB	Divide A by B
DA A	Decimal adjust A

SILICON LABS

ADD A,<source-byte> ADDC A,<source-byte>

- ADD adds the data byte specified by the source operand to the accumulator, leaving the result in the accumulator
- ADDC adds the data byte specified by the source operand, the carry flag and the accumulator contents, leaving the result in the accumulator
- Operation of both the instructions, ADD and ADDC, can affect the carry flag (CY), auxiliary carry flag (AC) and the overflow flag (OV)
  - CY=1 If there is a carryout from bit 7; cleared otherwise
  - AC =1 If there is a carryout from the lower 4-bit of A i.e. from bit 3; cleared otherwise
  - OV=1 If the signed result cannot be expressed within the number of bits in the destination operand; cleared otherwise

SUBB A,<source-byte>

- SUBB subtracts the specified data byte and the carry flag together from the accumulator, leaving the result in the accumulator

  CY=1 If a borrow is needed for bit 7; cleared otherwise

  AC =1 If a borrow is needed for bit 3, cleared otherwise

  OV=1 If a borrow is needed into bit 6, but not into bit 7, or into bit 7, but not into bit 6.

- *Example*:

  The accumulator holds 0C1H (11000001B), Register1 holds 40H (01000000B) and the CY=1.The instruction,

```
SUBB   A, R1
```

  gives the value 70H (01110000B) in the accumulator, with the CY=0 and AC=0 but OV=1

INC <byte>

- Increments the data variable by 1. The instruction is used in register, direct or register direct addressing modes
- *Example*:

```
INC6FH
```

  If the internal RAM location 6FH contains 30H, then the instruction increments this value, leaving 31H in location 6FH

- *Example*:

```
MOVR1, #5E
INCR1
INC@R1
```

- If R1=5E (01011110) and internal RAM location 5FH contains 20H, the instructions will result in R1=5FH and internal RAM location 5FH to increment by one to 21H

DEC <byte>

- The data variable is decremented by 1
- The instruction is used in accumulator, register, direct or register direct addressing modes
- A data of value 00H underflows to FFH after the operation
- No flags are affected

INC DPTR

- Increments the 16-bit data pointer by 1
- DPTR is the only 16-bit register that can be incremented
- The instruction adds one to the contents of DPTR directly

MUL AB

- Multiplies A & B and the 16-bit result stored in [B15-8], [A7-0]
- Multiplies the unsigned 8-bit integers in the accumulator and the B register
- The **Low** order byte of the 16-bit product will go to the accumulator and the **High** order byte will go to the B register
- If the product is greater than 255 (FFH), the overflow flag is set; otherwise it is cleared. The carry flag is always cleared.
- If ACC=85 (55H) and B=23 (17H), the instruction gives the product 1955 (07A3H), so B is now 07H and the accumulator is A3H. The overflow flag is set and the carry flag is cleared.

SILICON LABS

DIV AB

- Divides A by B
- THE INTEGER PART OF THE QUOTIENT IS STORED IN A AND THE REMAINDER GOES TO THE B REGISTER
- If ACC=90 (5AH) and B=05(05H), the instruction leaves 18 (12H) in ACC and the value 00 (00H) in B, since 90/5 = 18 (quotient) and 00 (remainder)
- Carry and OV are both cleared
- *If B contains 00H before the division operation, then the values stored in ACC and B are undefined and an overflow flag is set. The carry flag is cleared.*

DA A

- This is a decimal adjust instruction
- It adjusts the 8-bit value in ACC resulting from operations like ADD or ADDC and produces two 4-bit digits (in packed Binary Coded Decimal (BCD) format)
- Effectively, this instruction performs the decimal conversion by adding 00H, 06H, 60H or 66H to the accumulator, depending on the initial value of ACC and PSW
- If ACC bits A3-0 are greater than 9 (xxxx1010-xxxx1111), or if AC=1, then a value 6 is added to the accumulator to produce a correct BCD digit in the lower order nibble
- If CY=1, because the high order bits A7-4 is now exceeding 9 (1010xxxx-1111xxxx), then these high order bits will be increased by 6 to produce a correct proper BCD in the high order nibble but not clear the carry

Logical Operations

Mnemonic	Description
ANL A, Rn	A = A & [Rn]
ANL A, direct	A = A & [direct memory]
ANL A,@Ri	A = A & [memory pointed to by Ri]
ANL A,#data	A= A & immediate data
ANL direct,A	[direct] = [direct] & A
ANL direct,#data	[direct] = [direct] & immediate data
ORL A, Rn	A = A OR [Rn]
ORL A, direct	A = A OR [direct]
ORL A,@Ri	A = A OR [@RI]
ORL A,#data	A = A OR immediate data
ORL direct,A	[direct] = [direct] OR A
ORL direct,#data	[direct] = [direct] OR immediate data
XRL A, Rn	A = A XOR [Rn]
XRL A, direct	A = A XOR [direct memory]
XRL A,@Ri	A = A XOR [@Ri]
XRL A,#data	A = A XOR immediate data
XRL direct,A	[direct] = [direct] XOR A
XRL direct,#data	[direct] = [direct] XOR immediate data
CLR A	Clear A
CPL A	Complement A
RL A	Rotate A left
RLC A	Rotate A left (through C)
RR A	Rotate A right
RRC A	Rotate A right (through C)
SWAP A	Swap nibbles

- Logical instructions perform Boolean operations (AND, OR, XOR, and NOT) on data bytes on a ***bit-by-bit*** basis

SILICON LABS

ANL <dest-byte>,<source-byte>

- This instruction performs the logical AND operation on the source and destination operands and stores the result in the destination variable
- No flags are affected
- *Example*:

```
ANLA,R2
```

If ACC=D3H (11010011) and R2=75H (01110101), the result of the instruction is ACC=51H (01010001)

- The following instruction is also useful when there is a need to mask a byte
- *Example*:

```
ANLP1,#10111001B
```

ORL <dest-byte>,<source-byte>

- This instruction performs the logical OR operation on the source and destination operands and stores the result in the destination variable

- No flags are affected

- *Example*:

  ```
  ORLA,R2
  ```

  If ACC=D3H (11010011) and R2=75H (01110101), the result of the instruction is ACC=F7H (11110111)

- *Example*:

  ```
  ORLP1,#11000010B
  ```

  This instruction sets bits 7, 6, and 1 of output Port 1

XRL <dest-byte>,<source-byte>

- This instruction performs the logical XOR (Exclusive OR) operation on the source and destination operands and stores the result in the destination variable

- No flags are affected

- *Example*:

  ```
  XRLA,R0
  ```

  If ACC=C3H (11000011) and R0=AAH (10101010), then the instruction results in ACC=69H (01101001)

- *Example*:

  ```
  XRLP1,#00110001
  ```

  This instruction complements bits 5, 4, and 0 of output Port 1

CLR A and CPL A

CLR A

- This instruction clears the accumulator (all bits set to 0)
- No flags are affected
- If ACC=C3H, then the instruction results in ACC=00H

CPL A

- This instruction logically complements each bit of the accumulator (one's complement)
- No flags are affected
- If ACC=C3H (11000011), then the instruction results in ACC=3CH (00111100)

RL A

- The 8 bits in the accumulator are rotated one bit to the left. Bit 7 is rotated into the bit 0 position.
- No flags are affected
- If ACC=C3H (11000011), then the instruction results in ACC=87H (10000111) with the carry unaffected

RLC A

- The instruction rotates the accumulator contents one bit to the left through the carry flag
- Bit 7 of the accumulator will move into carry flag and the original value of the carry flag will move into the Bit 0 position
- No other flags are affected
- If ACC=C3H (11000011), and the carry flag is 1, the instruction results in ACC=87H (10000111) with the carry flag set

RR A

- The 8 bits in the accumulator are rotated one bit to the right. Bit 0 is rotated into the bit 7 position.
- No flags are affected
- If ACC=C3H (11000011), then the instruction results in ACC=E1H (11100001) with the carry unaffected

RRC A

- The instruction rotates the accumulator contents one bit to the right through the carry flag
- The original value of carry flag will move into Bit 7 of the accumulator and Bit 0 rotated into carry flag
- No other flags are affected
- If ACC=C3H (11000011), and the carry flag is 0, the instruction results in ACC=61H (01100001) with the carry flag set

SWAP A

- This instruction interchanges the low order 4-bit nibbles (A3-0) with the high order 4-bit nibbles (A7-4) of the ACC
- The operation can also be thought of as a 4-bit rotate instruction
- No flags are affected
- If ACC=C3H (11000011), then the instruction leaves ACC=3CH (00111100)

Data Transfer Instructions

Mnemonic	Description
MOV @Ri, direct	[@Ri] = [direct]
MOV @Ri, #data	[@Ri] = immediate data
MOV DPTR, #data 16	[DPTR] = immediate data
MOVC A,@A+DPTR	A = Code byte from [@A+DPTR]
MOVC A,@A+PC	A = Code byte from [@A+PC]
MOVX A,@Ri	A = Data byte from external ram [@Ri]
MOVX A,@DPTR	A = Data byte from external ram [@DPTR]
MOVX @Ri, A	External[@Ri] = A
MOVX @DPTR,A	External[@DPTR] = A
PUSH direct	Push into stack
POP direct	Pop from stack
XCH A,Rn	A = [Rn], [Rn] = A
XCH A, direct	A = [direct], [direct] = A
XCH A, @Ri	A = [@Rn], [@Rn] = A
XCHD A,@Ri	Exchange low order digits

MOV <dest-byte>,<source-byte>

- This instruction moves the source byte into the destination location
- The source byte is not affected, neither are any other registers or flags
- *Example*:

```
MOVR1,#60;R1=60H
MOVA,@R1      ;A=[60H]
MOVR2,#61;R2=61H
ADDA,@R2      ;A=A+[61H]
MOVR7,A       ;R7=A
```

- If internal RAM locations 60H=10H, and 61H=20H, then after the operations of the above instructions R7=A=30H. The data contents of memory locations 60H and 61H remain intact.

MOV DPTR, #data 16

- This instruction loads the data pointer with the 16-bit constant and no flags are affected

- *Example*:

```
MOVDPTR,#1032
```

- This instruction loads the value 1032H into the data pointer, i.e. DPH=10H and DPL=32H.

MOVC A,@A + <base-reg>

- This instruction moves a code byte from program memory into ACC
- The effective address of the byte fetched is formed by adding the original 8-bit accumulator contents and the contents of the base register, which is either the data pointer (DPTR) or program counter (PC)
- 16-bit addition is performed and no flags are affected
- The instruction is useful in reading the look-up tables in the program memory
- If the PC is used, it is incremented to the address of the following instruction before being added to the ACC
- *Example*:

```
        CLRA
LOC1:     INCA
        MOVC  A,@A + PC
        RET
Look_up      DB 10H
        DB 20H
        DB 30H
        DB 40H
```

- The subroutine takes the value in the accumulator to 1 of 4 values defined by the DB (define byte) directive
- After the operation of the subroutine it returns ACC=20H

MOVX <dest-byte>,<source-byte>

- This instruction transfers data between ACC and a byte of external data memory
- There are two forms of this instruction, the only difference between them is whether to use an 8-bit or 16-bit indirect addressing mode to access the external data RAM
- The 8-bit form of the MOVX instruction uses the EMI0CN SFR to determine the upper 8 bits of the effective address to be accessed and the contents of R0 or R1 to determine the lower 8 bits of the effective address to be accessed
- *Example*:

```
MOV EMI0CN,#10H   ;Load high byte of
;address into EMI0CN.
MOV R0,#34H    ;Load low byte of
;address into R0(or R1).
MOVX   A,@R0      ;Load contents of 1034H
 ;into ACC.
```

MOVX <dest-byte>,<source-byte>

- The 16-bit form of the MOVX instruction accesses the memory location pointed to by the contents of the DPTR register

- *Example*:

```
MOVDPTR,#1034H ;Load DPTR with 16 bit
          ;address to read (1034H).
MOVX  A,@DPTR  ;Load contents of 1034H
;into ACC.
```

- The above example uses the 16-bit immediate MOV DPTR instruction to set the contents of DPTR

- Alternately, the DPTR can be accessed through the SFR registers DPH, which contains the upper 8 bits of DPTR, and DPL, which contains the lower 8 bits of DPTR

PUSH Direct

- This instruction INCREMENTS THE STACK POINTER (SP) BY 1
- The contents of *Direct*, which is an internal memory location or a SFR, are copied into the internal RAM location addressed by the stack pointer
- No flags are affected
- *Example*:

```
PUSH   22H
PUSH   23H
```

- Initially the SP points to memory location 4FH and the contents of memory locations 22H and 23H are 11H and 12H respectively. After the above instructions, SP=51H, and the internal RAM locations 50H and 51H will store 11H and 12H respectively.

POP Direct

- This instruction reads the contents of the internal RAM location addressed by the stack pointer (SP) and decrements the stack pointer by 1. The data read is then transferred to the *Direct* address which is an internal memory or a SFR. No flags are affected.

- *Example*:

```
POPDPH
POPDPL
```

- If SP=51H originally and internal RAM locations 4FH, 50H and 51H contain the values 30H, 11H and 12H respectively, the instructions above leave SP=4FH and DPTR=1211H

```
POPSP
```

- If the above line of instruction follows, then SP=30H. In this case, SP is decremented to 4EH before being loaded with the value popped (30H)

XCH A,<byte>

- This instruction swaps the contents of ACC with the contents of the indicated data byte

- *Example*:

```
XCHA,@R0
```

- Suppose R0=2EH, ACC=F3H (11110011) and internal RAM location 2EH=76H (01110110). The result of the above instruction leaves RAM location 2EH=F3H and ACC=76H.

XCHD A,@Ri

- This instruction exchanges the low order nibble of ACC (bits 0-3), with that of the internal RAM location pointed to by Ri register
- The high order nibbles (bits 7-4) of both the registers remain the same
- No flags are affected
- Example:

```
XCHDA,@R0
```

If R0=2EH, ACC=76H (01110110) and internal RAM location 2EH=F3H (11110011), the result of the instruction leaves RAM location 2EH=F6H (11110110) and ACC=73H (01110011)

Boolean Variable Instructions

- The C8051F020 processor can perform single bit operations
- The operations include *set, clear*, as well as *and, or* and *complement* instructions
- Also included are bit–level moves or conditional jump instructions
- All bit accesses use direct addressing

Mnemonic	Description
CLR C	Clear C
CLR bit	Clear direct bit
SETB C	Set C
SETB bit	Set direct bit
CPL C	Complement c
CPL bit	Complement direct bit
ANL C,bit	AND bit with C
ANL C,/bit	AND NOT bit with C
ORL C,bit	OR bit with C
ORL C,/bit	OR NOT bit with C
MOV C,bit	MOV bit to C
MOV bit,C	MOV C to bit
JC rel	Jump if C set
JNC rel	Jump if C not set
JB bit,rel	Jump if specified bit set
JNB bit,rel	Jump if specified bit not set
JBC bit,rel	if specified bit set then clear it and jump

CLR <bit>

- This operation clears (reset to 0) the specified bit indicated in the instruction
- No other flags are affected
- CLR instruction can operate on the carry flag or any directly-addressable bit
- *Example*:

```
CLRP2.7
```

  If Port 2 has been previously written with DCH (11011100), then the operation leaves the port set to 5CH (01011100)

SETB <bit>

- This operation sets the specified bit to 1
- SETB instruction can operate on the carry flag or any directly-addressable bit
- No other flags are affected
- *Example*:

```
SETB   C
SETB   P2.0
```

- If the carry flag is cleared and the output Port 2 has the value of 24H (00100100), then the result of the instructions sets the carry flag to 1 and changes the Port 2 value to 25H (00100101)

CPL <bit>

- This operation complements the bit indicated by the operand
- No other flags are affected
- CPL instruction can operate on the carry flag or any directly-addressable bit
- *Example*:

```
CPLP2.1
CPLP2.2
```

- If Port 2 has the value of 53H (01010011) before the start of the instructions, then after the execution of the instructions it leaves the port set to 55H (01010101)

ANL C, <source-bit>

- This instruction ANDs the bit addressed with the Carry bit and stores the result in the Carry bit itself
- If the source bit is a logical 0, then the instruction clears the carry flag; else the carry flag is left in its original value
- If a slash (/) is used in the source operand bit, it means that the logical complement of the addressed source bit is used, ***but the source bit itself is not affected***
- No other flags are affected
- *Example*:

```
MOVC,P2.0;Load C with input pin
;state of P2.0.
ANLC,P2.7;AND carry flag with                    ;bit 7
of P2.
MOVP2.1,C;Move C to bit 1 of Port 2.
ANLC,/OV     ;AND with inverse of OV flag.
```

- If P2.0=1, P2.7=0 and OV=0 initially, then after the above instructions, P2.1=0, CY=0 and the OV remains unchanged, i.e. OV=0

ORL C, <source-bit>

- This instruction ORs the bit addressed with the Carry bit and stores the result in the Carry bit itself
- It sets the carry flag if the source bit is a logical 1; else the carry is left in its original value
- If a slash (/) is used in the source operand bit, it means that the logical complement of the addressed source bit is used, ***but the source bit itself is not affected***
- No other flags are affected
- *Example*:

```
MOV C,P2.0      ;Load C with input pin
;state of P2.0.
ORL C,P2.7      ;OR carry flag with
;bit 7 of P2.
MOV P2.1,C      ;Move C to bit 1 of
;port 2.
ORL C,/OV       ;OR with inverse of OV
;flag.
```

MOV <dest-bit>,<source-bit>

- The instruction loads the value of source operand bit into the destination operand bit
- One of the operands **must** be the carry flag; the other may be any directly-addressable bit
- No other register or flag is affected
- *Example*:

```
MOVP2.3,C
MOVC,P3.3
MOVP2.0,C
```

- If P2=C5H (11000101), P3.3=0 and CY=1 initially, then after the above instructions, P2=CCH (11001100) and CY=0.

JC rel

- This instruction branches to the address, indicated by the label, if the carry flag is set, otherwise the program continues to the next instruction
- No flags are affected
- *Example*:

```
CLRC
SUBB  A,R0
JC ARRAY1
MOVA,#20H
```

- The carry flag is cleared initially. After the SUBB instruction, if the value of A is smaller than R0, then the instruction sets the carry flag and causes program execution to branch to ARRAY1 address, otherwise it continues to the MOV instruction.

JNC rel

- This instruction branches to the address, indicated by the label, if the carry flag is **not** set, otherwise the program continues to the next instruction

- No flags are affected. **The carry flag is not modified.**

- *Example*:

```
CLRC
SUBB   A,R0
JNCARRAY2
MOVA,#20H
```

- The above sequence of instructions will cause the jump to be taken if the value of A is greater than or equal to R0. Otherwise the program will continue to the MOV instruction.

JB <bit>,rel

- This instruction jumps to the address indicated if the destination bit is 1, otherwise the program continues to the next instruction

- No flags are affected. **The bit tested is not modified**.

- *Example*:

```
JB ACC.7,ARRAY1
JB P1.2,ARRAY2
```

- If the accumulator value is 01001010 and Port 1=57H (01010111), then the above instruction sequence will cause the program to branch to the instruction at ARRAY2

JNB <bit>,rel

- This instruction jumps to the address indicated if the destination bit is 0, otherwise the program continues to the next instruction

- No flags are affected. **The bit tested is not modified**.

- *Example*:

```
JNBACC.6,ARRAY1
JNBP1.3,ARRAY2
```

- If the accumulator value is 01001010 and Port 1=57H (01010111), then the above instruction sequence will cause the program to branch to the instruction at ARRAY2

JBC <bit>,rel

- If the source bit is 1, this instruction clears it and branches to the address indicated; else it proceeds with the next instruction

- **The bit is not cleared if it is already a 0**. No flags are affected.

- *Example*:

```
JBCP1.3,ARRAY1
JBCP1.2,ARRAY2
```

- If P1=56H (01010110), the above instruction sequence will cause the program to branch to the instruction at ARRAY2, modifying P1 to 52H (01010010)

Program Branching Instructions

- Program branching instructions are used to control the flow of actions in a program
- Some instructions provide decision making capabilities and transfer control to other parts of the program, e.g. conditional and unconditional branches

Mnemonic	Description
ACALL addr11	Absolute subroutine call
LCALL addr16	Long subroutine call
RET	Return from subroutine
RETI	Return from interrupt
AJMP addr11	Absolute jump
LJMP addr16	Long jump
SJMP rel	Short jump
JMP @A+DPTR	Jump indirect
JZ rel	Jump if A=0
JNZ rel	Jump if A NOT=0
CJNE A,direct,rel	Compare and Jump if Not Equal
CJNE A,#data,rel	
CJNE Rn,#data,rel	
CJNE @Ri,#data,rel	
DJNZ Rn,rel	Decrement and Jump if Not Zero
DJNZ direct,rel	
NOP	No Operation

ACALL addr11

- This instruction **unconditionally** calls a subroutine indicated by the address
- The operation will cause the PC to increase by 2, then it pushes the 16-bit PC value onto the stack (low order byte first) and increments the stack pointer twice
- The PC is now loaded with the value *addr11* and the program execution continues from this new location
- The subroutine called must therefore start within the same 2 kB block of the program memory

- No flags are affected

- *Example*:

```
ACALL LOC_SUB
```

- If SP=07H initially and the label “LOC_SUB” is at program memory location 0567H, then executing the instruction at location 0230H, SP=09H, internal RAM locations 08H and 09H will contain 32H and 02H respectively and PC=0567H

LCALL addr16

- This instruction calls a subroutine located at the indicated address
- The operation will cause the PC to increase by 3, then it pushes the 16-bit PC value onto the stack (low order byte first) and increments the stack pointer twice
- The PC is then loaded with the value *addr16* and the program execution continues from this new location
- Since it is a Long call, the subroutine may therefore begin anywhere in the full 64 kB program memory address space
- No flags are affected
- *Example*:

```
LCALL LOC_SUB
```

- Initially, SP=07H and the label “LOC_SUB” is at program memory location 2034H. Executing the instruction at location 0230H, SP=09H, internal RAM locations 08H and 09H contain 33H and 02H respectively and PC=2034H

RET

- This instruction returns the program from a subroutine
- RET pops the high byte and low byte address of PC from the stack and decrements the SP by 2
- The execution of the instruction will result in the program to resume from the location just after the “call” instruction
- No flags are affected
- Suppose SP=0BH originally and internal RAM locations 0AH and 0BH contain the values 30H and 02H respectively. The instruction leaves SP=09H and program execution will continue at location 0230H

RETI

- This instruction returns the program from an interrupt subroutine
- RETI pops the high byte and low byte address of PC from the stack and restores the interrupt logic to accept additional interrupts
- SP decrements by 2 and no other registers are affected. However the PSW is not automatically restored to its pre-interrupt status
- After the RETI, program execution will resume immediately after the point at which the interrupt is detected
- Suppose SP=0BH originally and an interrupt is detected during the instruction ending at location 0213H
  - Internal RAM locations 0AH and 0BH contain the values 14H and 02H respectively
  - The RETI instruction leaves SP=09H and returns program execution to location 0234H

AJMP addr11

- The AJMP instruction transfers program execution to the destination address which is located at the absolute short range distance (short range means 11-bit address)
- The destination must therefore be within the same 2kB block of program memory
- *Example*:

```
AJMP   NEAR
```

- If the label NEAR is at program memory location 0120H, the AJMP instruction at location 0234H loads the PC with 0120H

LJMP addr16

- The LJMP instruction transfers program execution to the destination address which is located at the absolute long range distance (long range means 16-bit address)
- The destination may therefore be anywhere in the full 64 kB program memory address space
- No flags are affected
- *Example*:

```
LJMP   FAR_ADR
```

- If the label FAR_ADR is at program memory location 3456H, the LJMP instruction at location 0120H loads the PC with 3456H

SJMP rel

- This is a short jump instruction, which increments the PC by 2 and then adds the relative value '*rel*' (signed 8-bit) to the PC
- This will be the new address where the program would branch to unconditionally
- Therefore, the range of destination allowed is from -128 to +127 bytes from the instruction
- *Example*:

```
SJMP   RELSRT
```

- If the label RELSRT is at program memory location 0120H and the SJMP instruction is located at address 0100H, after executing the instruction, PC=0120H.

JMP @A + DPTR

- This instruction adds the 8-bit unsigned value of the ACC to the 16-bit data pointer and the resulting sum is returned to the PC
- Neither ACC nor DPTR is altered
- No flags are affected
- *Example*:

```
        MOVDPTR, #LOOK_TBL
        JMP@A + DPTR
LOOK_TBL:AJMP   LOC0
        AJMP   LOC1
        AJMP   LOC2
```

  If the ACC=02H, execution jumps to LOC1

- AJMP is a two byte instruction

JZ rel

- This instruction branches to the destination address if ACC=0; else the program continues to the next instruction
- The ACC is not modified and no flags are affected
- *Example*:

```
SUBB A,#20H
JZ LABEL1
DECA
```

- If ACC originally holds 20H and CY=0, then the SUBB instruction changes ACC to 00H and causes the program execution to continue at the instruction identified by LABEL1; otherwise the program continues to the DEC instruction

JNZ rel

- This instruction branches to the destination address if any bit of ACC is a 1; else the program continues to the next instruction
- The ACC is not modified and no flags are affected
- *Example*:

```
DECA
JNZLABEL2
MOVRO, A
```

- If ACC originally holds 00H, then the instructions change ACC to FFH and cause the program execution to continue at the instruction identified by LABEL2; otherwise the program continues to MOV instruction

CJNE <dest-byte>,<source-byte>,rel

- This instruction compares the magnitude of the *dest-byte* and the *source-byte* and branches if their values are not equal
- The carry flag is set if the unsigned *dest-byte* is less than the unsigned integer *source-byte*; otherwise, the carry flag is cleared
- Neither operand is affected
- *Example*:

```
CJNE   R3,#50H,NEQU
   … …          ;R3 = 50H
NEQU: JC LOC1       ;If R3 < 50H
   … …          ;R7 > 50H
LOC1: … …          ;R3 < 50H
```

DJNZ <byte>,<rel-addr>

- This instruction is ”decrement jump not zero”
- It decrements the contents of the destination location and if the resulting value is not 0, branches to the address indicated by the source operand
- An original value of 00H underflows to FFH
- No flags are affected

- *Example*:

```
DJNZ 20H,LOC1
DJNZ 30H,LOC2
DJNZ 40H,LOC3
```

- If internal RAM locations 20H, 30H and 40H contain the values 01H, 5FH and 16H respectively, the above instruction sequence will cause a jump to the instruction at LOC2, with the values 00H, 5EH, and 15H in the 3 RAM locations.
  - Note, the first instruction will not branch to LOC1 because the [20H] = 00H, hence the program continues to the second instruction
  - Only after the execution of the second instruction (where the location [30H] = 5FH), then the branching takes place

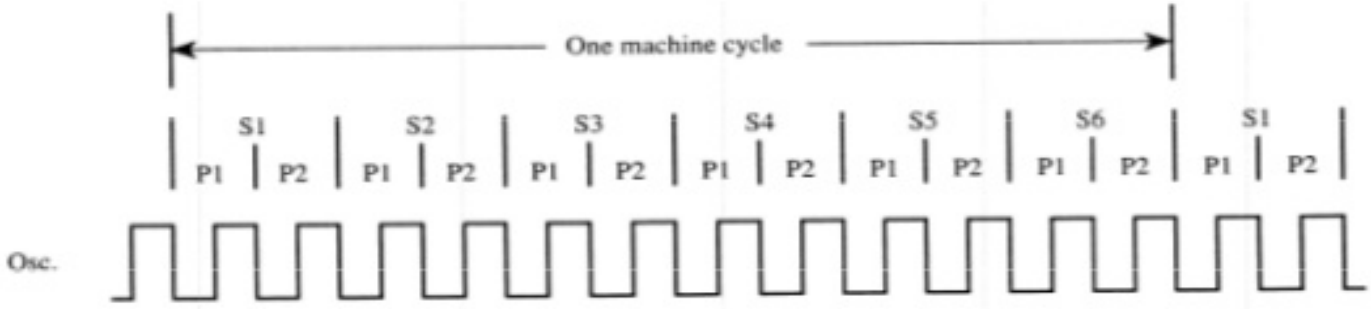
NOP

- This is the no operation instruction
- The instruction takes one machine cycle operation time
- Hence it is useful to time the ON/OFF bit of an output port
- *Example*:

```
CLRP1.2
NOP
NOP
NOP
NOP
SETB   P1.2
```

- The above sequence of instructions outputs a low-going output pulse on bit 2 of Port 1 lasting exactly 5 cycles.
  - Note a simple SETB/CLR generates a 1 cycle pulse, so four additional cycles must be inserted in order to have a 5-clock pulse width

Concept of Machine Cycle



- Time Period of 1 clock cycle = 1/f = 1/12 microsec.
  - **1 T State = 2 clock cycles**
- Time Period of 1 T State = 2 x 1/12 = 1/6 microsec.
  - **1 Machine Cycle = 6 T States**
- Time Period of 1 Machine Cycle = 6 x 1/6 = 1 microsec.

65

copyright@gauravverma

- However, the relation between machine cycle and clock cycle depends upon the manufacturer. Ex: AT89C51 by Atmel consider 12 clock cycles per machine cycle. But, other 8051 manufactured by Dallas consider 1 clock cycles per machine cycle.

Instruction cycle: Fetch, decode, execute, write back into memory and etc.

ARITHMETIC INSTRUCTIONS

Instruction opcode	Operand	Size in bytes	machine cycles
ADD	A,Rn	1	1 (12 Oscillator periods)
ADDC	A,Rn	1	1
SUBB	A,Rn	1	1
INC	A	1	1
DEC	A	1	1
INC	DPTR	1	2
MUL	AB	1	4 (i.e. 48 Oscillator periods)
DIV	AB	1	4 (i.e. Oscillator periods)

- **Note:** Mnemonic will occupy 1 byte memory.
- No register will occupy memory of RAM or ROM.
- Address will occupy memory depending upon the number of bits.
- Data also occupy memory depending upon number of bits.

LOGICAL INSTRUCTIONS

Instruction opcode	Operand	Size in bytes	machine cycles
ANL	A,Rn	1	1
ANL	A,direct	2	1
ANL	A,@Ri	1	1
ANL	A,#data	2	1
ANL	direct,A	2	1
ANL	direct,#data	3	2
ORL	A,Rn	1	1
XRL	A,Rn	1	1
CLR	A	1	1
CPL	A	1	1
RL	A	1	1
RLC	A	1	1
RR	A	1	1

DATA TRANSFER INSTRUCTIONS

Instruction opcode	Operand	Size in bytes	machine cycles
MOV	A,Rn	1	1
MOV	A,direct	2	1
MOV	A,#data	2	1
MOVC	A,@A+DPTR	1	2
MOVX	A,@Ri	1	2
PUSH	direct	2	2
POP	direct	2	2
XCH	A,Rn	1	1
XCH	A,direct	2	1
XCH	A,@Ri	1	1
XCHD	A,@Ri	1	1

BOOLEAN INSTRUCTIONS

Instruction opcode	Operand	Size in bytes	machine cycles
CLR	C	1	1
CLR	bit	2	1
SETB	C	1	1
SETB	bit	2	1
CPL	C	1	1
CPL	bit	2	1
ANL	C,bit	2	2
ANL	C,/bit	2	2
ORL	C,bit	2	2
MOV	C,bit	2	1
MOV	bit,C	2	2
JC	rel	2	2
JNC	rel	2	2
JNC	rel	2	2
JB	bit,rel	3	2
JNB	bit,rel	3	2

BRANCHING INSTRUCTIONS

Instruction opcode	Operand	Size in bytes	machine cycles
ACALL	addr11	2	2
LCALL	addr16	3	2
RET	-	1	2
RETI	-	1	2
AJMP	addr11	2	2
LJMP	addr16	3	2
JMP	@A+DPTR	1	2
JZ	rel	2	2
JNZ	rel	2	2
CJNE	A,direct,rel	3	2
DJNZ	Rn,rel	2	2
DJNZ	direct,rel	3	2

Example 3-13

The following shows crystal frequency for three different 8051-based systems. Find the period of the machine cycle in each case.
(a) 11.0592 MHz (b) 16 MHz (c) 20 MHz

Solution:

(a) 11.0592 MHz/12 = 921.6 kHz; machine cycle is 1/921.6 kHz = 1.085 μs (microsecond)
(b) 16 MHz/12 = 1.333 MHz; machine cycle (MC) = 1/1.333 MHz = 0.75 μs
(c) 20 MHz/12 = 1.66 MHz; MC = 1/1.66 MHz = 0.60 μs

Example 3-14

For an 8051 system of 11.0592 MHz, find how long it takes to execute each of the following instructions.

(a) `MOV R3,#55` (b) `DEC R3` (c) `DJNZ R2,target`
(d) `LJMP` (e) `SJMP` (f) `NOP` (no operation)
(g) `MUL AB`

Solution:

The machine cycle for a system of 11.0592 MHz is 1.085 µs as shown in Example 3-13. Table A-1 in Appendix A shows machine cycles for each of the above instructions. Therefore, we have:

Instruction	*Machine cycles*	*Time to execute*
(a) `MOV R3,#55`	1	1×1.085 µs = 1.085 µs
(b) `DEC R3`	1	1×1.085 µs = 1.085 µs
(c) `DJNZ R2,target`	2	2×1.085 µs = 2.17 µs
(d) `LJMP`	2	2×1.085 µs = 2.17 µs
(e) `SJMP`	2	2×1.085 µs = 2.17 µs
(f) `NOP`	1	1×1.085 µs = 1.085 µs
(g) `MUL AB`	4	4×1.085 µs = 4.34 µs

Delay calculation for 8051

As seen in the last section, a delay subroutine consists of two parts: (1) setting a counter, and (2) a loop. Most of the time delay is performed by the body of the loop, as shown in Example 3-15.

Example 3-15

Find the size of the delay in the following program, if the crystal frequency is 11.0592 MHz.

```
          MOV  A,#55H     ;load A with 55H
AGAIN:    MOV  P1,A       ;issue value in reg A to port 1
          ACALL DELAY     ;time delay
          CPL  A          ;complement reg A
          SJMP AGAIN      ;keep doing this indefinitely
;----Time delay
DELAY:    MOV  R3,#200    ;load R3 with 200
HERE:     DJNZ R3,HERE    ;stay here until R3 become 0
          RET             ;return to caller
```

Solution:

From Table A-1 in Appendix A, we have the following machine cycles for each instruction of the DELAY subroutine.

```
                          Machine Cycle
DELAY:    MOV  R3,#200         1
HERE:     DJNZ R3,HERE         2
          RET                  2
```

Therefore, we have a time delay of $[(200 \times 2) + 1 + 2] \times 1.085\ \mu s = 436.255\ \mu s$.

Loop inside loop delay

Another way to get a large delay is to use a loop inside a loop, which is also called a *nested loop*. See Example 3-17.

For an 8051 system of 11.0592 MHz, find the time delay for the following subroutine:

```
                              Machine Cycle
DELAY:    MOV   R3,#250           1

HERE:     NOP                     1
          NOP                     1
          NOP                     1
          NOP                     1
          DJNZ  R3,HERE           2

          RET                     2
```

Example 3-17

For a machine cycle of 1.085 μs, find the time delay in the following subroutine.

```
DELAY:                        Machine Cycle
          MOV   R2,#200           1
AGAIN:    MOV   R3,#250           1
HERE:     NOP                     1
          NOP                     1
          DJNZ  R3,HERE           2
          DJNZ  R2,AGAIN          2
          RET                     2
```

For the HERE loop, we have (4 x 250) xl.085 us = 1085 us. The AGAIN loop repeats the HERE loop 200 times; therefore, we have 200 x 1085 us = 217000, if we do not include the overhead. However, the instructions "MOV R3,#250" and "DJNZ R2, AGAIN" at the beginning and end of the AGAIN loop add (3 x 200 x 1.085 us) = 651 us to the time delay. As a result we have 217000 + 651 = 217651 us = 217.651 milliseconds for total time delay associated with the above DELAY subroutine. Notice that in the case of a nested loop, as in all other time delay loops, the time is approximate since we have ignored the first and last instructions in the subroutine.

Example 3-21

Write a program to toggle all the bits of P1 every 200 ms. Assume that the crystal frequency is 11.0592 MHz, and that the system is using the AT89C51.

Solution:

```
;Tested for AT89C51 of 11.0592 MHz.

          MOV   A,#55H
AGAIN:    MOV   P1,A
          ACALL DELAY
          CPL   A
          SJMP  AGAIN

;----Time delay
DELAY:    MOV   R5,#2
HERE1:    MOV   R4,#180
HERE2:    MOV   R3,#255
HERE3:    DJNZ  R3,HERE3
          DJNZ  R4,HERE2
          DJNZ  R5,HERE1
          RET
```

$2 \times 180 \times 255 \times 2$ MC $\times$ 1.085 µs = 199,206 µs